

第八周作业 -2- 提高作业

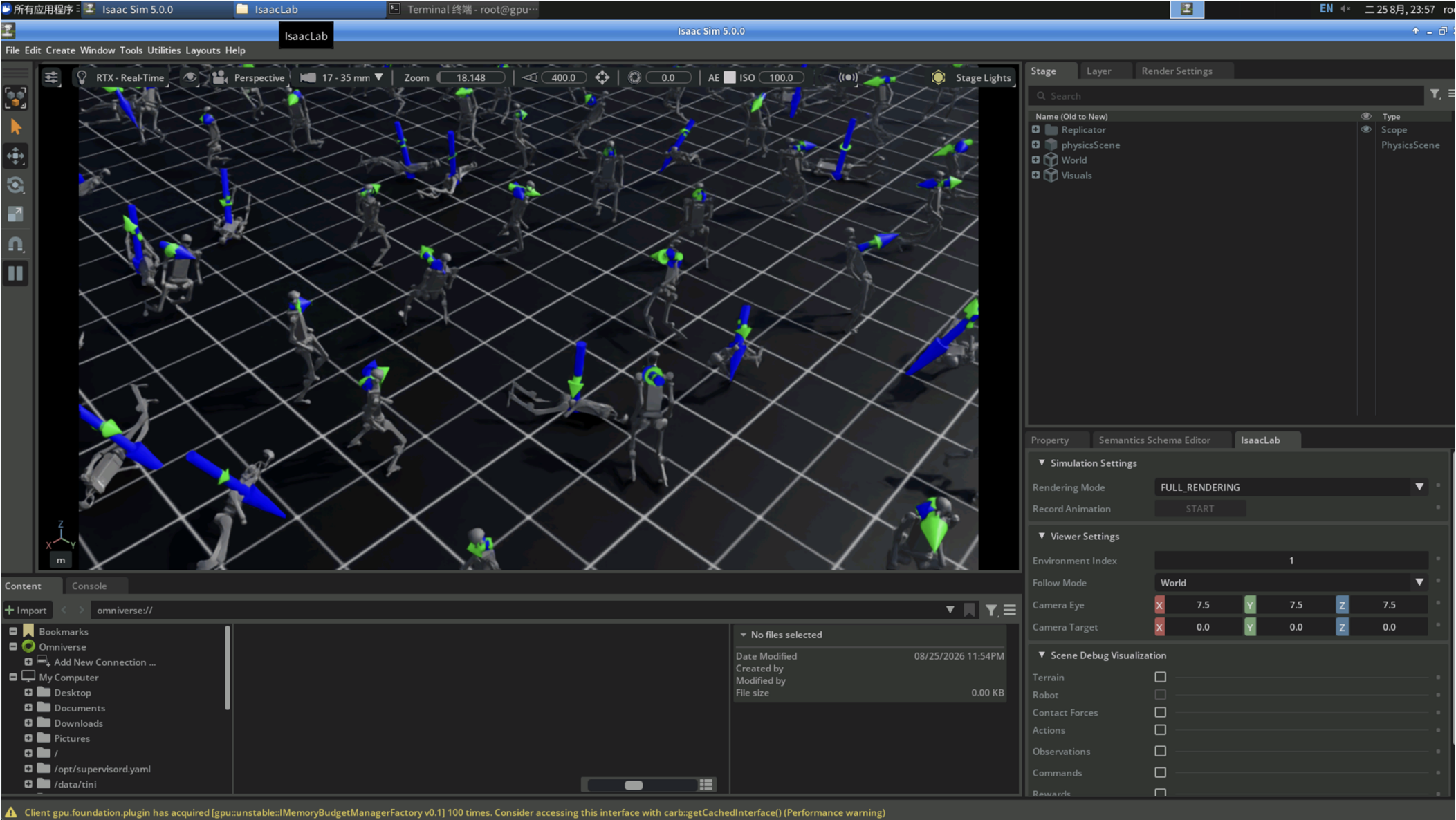
第八周作业 -2- 提高作业

2.1 选择一个 locomotion 任务

- 在官方 locomotion 任务中，自选 1 个任务进行分析；
- Isaac-Humanoid-v0 Isaac-Humanoid-v0，建议自行选择另外一个 locomotion 环境；

以下以 locomotion 任务为例进行

```
./isaacsim.sh -p scripts/reinforcement_learning/rsl_rl/train.py --task=Isaac-Velocity-Flat-H1-v0
```



2.2 Reward 设计分析

- 找到该任务对应的 reward 定义位置，完成以下分析：
- 列出该任务的所有 reward 项；
 - 对每个 reward 项分别说明：
 - 权重是多少；
 - 它奖励什么行为；
 - 它惩罚什么行为；
 - 总结这个任务的 reward 设计逻辑；
 - 分析这个 reward 设计可能仍存在什么不足，谈谈你的理解。
建议整理成类似表格：

reward 项	原始权重	奖励或惩罚什么	你的理解
----------	------	---------	------

宇树 H1 平地行走任务 Reward 分析

1. 全部 Reward 项一览表

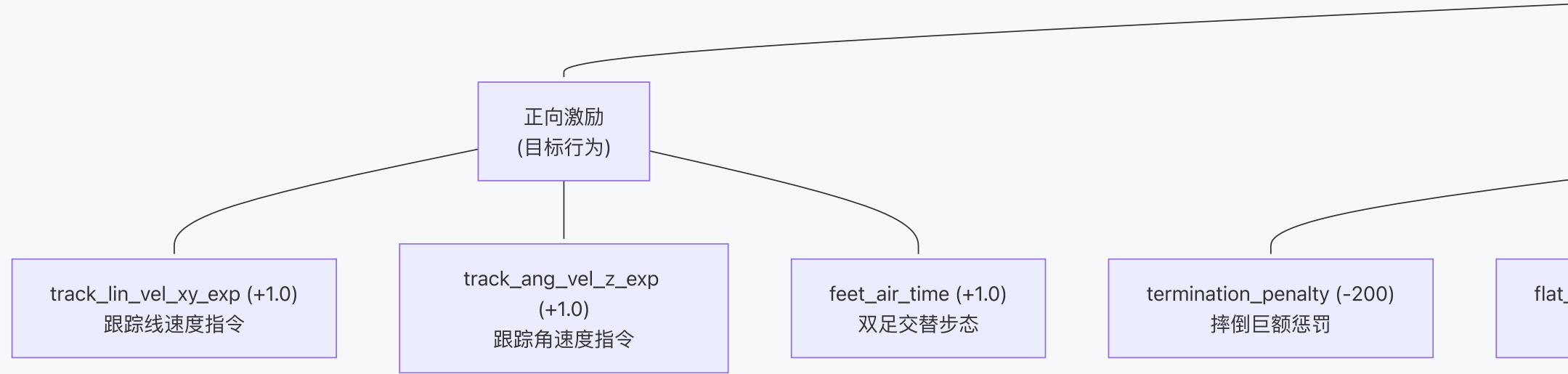
#	Reward 项	函数	权重	奖励 / 惩罚什么	理解与分析
1	track_lin_vel_xy_exp	track_lin_vel_xy_yaw_frame_exp	1	奖励：在 yaw 对齐的机体坐标系中，实际 xy 线速度与指令线速度之间的误差越小奖励越高（使用高斯指数核 $\exp(-error/\sigma)$ ， $\sigma=0.5$ ）	核心正向奖励之一。指数核使得在误差为 0 时奖励为 1，误差越大指数衰减，比 L2 惩罚更平滑，鼓励精确跟踪速度指令
2	track_ang_vel_z_exp	track_ang_vel_z_world_exp	1	奖励：实际 yaw 角速度与指令角速度之间的误差越小奖励越高（同样使用指数核， $\sigma=0.5$ ）	核心正向奖励之一。确保机器人能精确转向。与线速度跟踪同等权重，说明设计者认为转向能力和直线行走同等重要
3	ang_vel_xy_l2	ang_vel_xy_l2	-0.05	惩罚：机体在 roll/pitch 方向的角速度（xy 轴角速度的 L2 平方）	抑制身体的左右摇晃和前后俯仰。权重较小（-0.05），仅作为软约束，避免过度干扰行走动态
4	dof_torques_l2	joint_torques_l2	0	惩罚：所有关节力矩的 L2 平方和（但权重为 0，未启用）	虽然定义了但权重为 0，说明当前训练不限制关节力矩。保留此项是为方便后续调参时启用
5	dof_acc_l2	joint_acc_l2	-1.25E-07	惩罚：所有关节加速度的 L2 平方和	极小权重的平滑项。抑制关节加速度突变，使动作过渡更平滑、减少高频抖动，有助于 sim-to-real 迁移
6	action_rate_l2	action_rate_l2	-0.005	惩罚：相邻两步动作的差值的 L2 平方和	鼓励策略输出平滑连续的动作，避免剧烈跳变。这对电机寿命和 sim-to-real 都很关键
7	feet_air_time	feet_air_time_positive_biped	1	奖励：双足交替抬脚（单脚站立阶段），脚在空中或触地的持续时间越接近阈值（0.6s）奖励越高。命令接近零时不给奖励	专为双足设计的步态奖励。强制 single stance 模式（同时只有一只脚着地），防止拖步或跳跃。阈值 0.6s 对应约 0.8~1.0 m/s 的步频
8	flat_orientation_l2	flat_orientation_l2	-1	惩罚：机体姿态偏离水平面的程度（投影重力在 xy 方向分量的 L2 平方）	较大惩罚权重（-1.0），强制保持躯干水平。对于高重心的 H1 人形机器人，这对稳定性至关重要
9	dof_pos_limits	joint_pos_limits	-1	惩罚：ankle 关节位置超出软限位的程度（超出部分的绝对值之和）	仅针对脚踝关节。防止脚踝达到机械极限，保护硬件并避免不自然的步态
10	termination_penalty	is_terminated	-200	惩罚：当 episode 因非法终止（如摔倒）而结束时，给予巨大负奖励	最大权重的惩罚项。摔倒代价极高，强烈引导策略学会保持平衡。不包含正常超时终止
11	feet_slide	feet_slide	-0.25	惩罚：脚在接触地面时的滑动速度（接触力>1N 时，脚部 xy 线速度的 norm 之和）	惩罚脚底打滑。鼓励脚在着地阶段保持静止，产生更稳定、更高效的步态
12	joint_deviation_hip	joint_deviation_l1	-0.2	惩罚：hip_yaw 和 hip_roll 关节偏离默认位置的 L1 距离	约束髋关节保持中立位，防止机器人出现外八字、内八字或躯干扭转等不自然姿态
13	joint_deviation_arms	joint_deviation_l1	-0.2	惩罚：所有肩关节和肘关节偏离默认位置的 L1 距离	让手臂保持默认姿态（自然下垂），防止策略利用手臂大幅摆动来辅助平衡，使步态更自然
14	joint_deviation_torso	joint_deviation_l1	-0.1	惩罚：torso（躯干旋转）关节偏离默认位置的 L1 距离	限制躯干扭转，权重稍小（-0.1），允许一定程度的躯干运动但不能过大

Note

`lin_vel_z_l2`（惩罚 z 轴线速度）和 `undesired_contacts`（惩罚非法接触）在配置中设为 `null`，未启用。

2. Reward 设计逻辑总结

2.1 总体架构



2.2 设计理念

- 3 项正向奖励 构成核心任务目标：速度跟踪 + 步态形成，三者权重均为 1.0，地位对等
- 层次化惩罚设计：
 - 生存级 (-200.0)：摔倒是最严重的失败
 - 姿态级 (-1.0)：保持水平 + 关节限位
 - 步态级 (-0.25)：防脚滑
 - 自然性级 (-0.1 ~ -0.2)：关节默认位约束
 - 平滑级 (-0.005 ~ -1.25e-7)：动作和加速度正则化
- 指数核 vs L2 核：速度跟踪使用指数核（有界 [0,1]，梯度友好），惩罚项使用 L2/L1 核（无界，越偏离惩罚越大）
- 双足专用设计：`feet_air_time_positive_biped` 强制 single stance，这是区别于四足的关键

3. 潜在不足分析

问题类别	具体不足	详细分析
z 轴运动无约束	<code>lin_vel_z_l2</code> 设为 null	未惩罚 z 轴线速度（弹跳），机器人可能学到在行走中产生不必要的上下弹跳，浪费能量且降低稳定性。在平地上问题不大，但迁移到真实场景可能暴露
能耗无约束	<code>dof_torques_l2</code> 权重为 0	完全不限制力矩大小，策略可能学到使用很大的力矩来实现目标，这对真实电机不友好。建议至少给一个小权重（如 1e-5）
步频/步幅不可控	仅靠 air_time 阈值间接调节	0.6s 的阈值固定了步频上限，但缺少对步幅的显式控制。不同速度指令可能需要不同的步频/步幅组合
无对称性奖励	左右腿无对称约束	双足行走理想情况下应该左右对称，但奖励中没有显式鼓励对称步态，可能导致学到一瘸一拐的步态
侧向运动能力弱	<code>lin_vel_y</code> 指令范围为 [0,0]	当前只训练前向行走（vx: 0~1 m/s, vy: 0），机器人不会学到侧向移动能力，泛化性受限

问题类别	具体不足	详细分析
无 body contact 惩罚	<code>undesired_contacts</code> 设为 null	除了 torso 碰地触发终止外，膝盖、手臂等部位碰地不会被惩罚，可能出现膝盖擦地等不良姿态
无基座高度约束	缺少 <code>base_height_l2</code>	H1 机器人重心较高（init 1.05m），没有高度奖励可能导致策略学到蹲着走（降低重心虽稳定但不自然）
手臂约束可能过强	arms deviation 权重 -0.2	自然的人类行走中手臂会有协调摆动来辅助平衡和效率。完全约束手臂不动可能限制了更优步态的探索
Domain Randomization 不足	外力/质量随机化范围为 0	<code>base_external_force_torque</code> 的 force_range 和 torque_range 都是 (0,0)， <code>add_base_mass</code> 为 null。这意味着训练时没有外界扰动，策略的鲁棒性可能不足
平地专用	地形为 plane	仅在完全平坦的地面上训练，缺少对不平整地面的适应能力。reward 中也没有对地形适应的相关项（如 height_scan 观测也是 null）

4. 改进建议优先级

 Tip

按从易到难、从高收益到低收益排序：

- 启用 `dof_torques_l2`（权重 ~1e-5）和 `lin_vel_z_l2`（权重 ~-0.5）— 低成本高收益
- 开启 **Domain Randomization** — 增加外力扰动和质量随机化，显著提升鲁棒性
- 添加 `base_height_l2` — 防止蹲行，保持自然站姿
- 拓展速度指令范围 — 加入侧向速度训练
- 考虑添加步态对称性奖励 — 提升步态自然度
- 适度放松手臂约束 — 允许自然摆臂，或引入与步态协调的手臂摆动奖励

2.3 修改 reward 权重并进行对比实验

在你选择的任务中，挑选 1 个 **reward** 项 做修改。

要求：

1. 原则上只改 1 个权重；

2. 改动幅度要足够明显，建议至少增加或减少 **50%**；

3. 在修改前，先写下你的预测：

- 你觉得训练会变得更稳还是更激进？
- 你觉得机器人会更快还是更保守？
- 你预计哪条曲线最明显变化？

H1 Reward 修改实验报告： `flat_orientation_l2` 权重 -1.0 → -0.25

修改文件: `rough_env_cfg.py`

1. 关键指标对比表

1.1 训练总览

指标	阶段	基线 (权重 -1.0)	修改版 (权重 -0.25)	变化
Mean Reward	iter 0	-0.157	-0.15	≈相同
	iter 10	-5.415	-5.08	↑ +6.2% (更好)
	iter 30	-5.107	-4.91	↑ +3.9%
	iter 49	-4.870	-4.88	↓ -0.2% (≈持平)
Episode Length	iter 0	13.1	13.1	≈相同
	iter 10	55.9	54.1	↓ -3.2%
	iter 30	73.2	78.0	↑ +6.6%
	iter 49	91.8	99.4	↑ +8.3% 

1.2 各 Reward 项对比 (iter 49)

Reward 项	基线 (权重 -1.0)	修改版 (权重 -0.25)	变化幅度	说明
<code>track_lin_vel_xy_exp</code>	0.0361	0.0390	↑ +8.0% 	速度跟踪更好
<code>track_ang_vel_z_exp</code>	0.0082	0.0094	↑ +14.6% 	转向跟踪也更好
<code>flat_orientation_l2</code>	-0.0062	-0.0030	↓ -51.6% 	权重降低，惩罚值更小（但实际倾斜角可能更大）
<code>termination_penalty</code>	-0.2000	-0.2000	无变化	两者都在 100% 碰地终止
<code>feet_air_time</code>	0.0009	0.0012	↑ +33.3% 	步态奖励提升
<code>feet_slide</code>	-0.0156	-0.0174	↓ -11.5%	略有更多脚滑
<code>error_vel_xy</code>	0.1335	0.1531	↑ +14.7%	速度误差略增（速度更高但偏差也稍大）

2. 训练曲线趋势分析

2.1 Episode Length（最显著变化）



修改版的 episode length 后期增长更快，在 iter 49 时比基线多活了 ~7.6 步。这意味着虽然姿态约束放松了，但机器人并没有更容易摔倒，反而活得更久。

2.2 flat_orientation_l2 惩罚



🔗 Important

关键发现：修改版的 `flat_orientation_l2` 惩罚绝对值只有基线的 **~48%**。但要注意：权重从 -1.0 降到 -0.25，如果机器人倾斜程度不变，惩罚值应降到 25%。实际降到 48% 意味着 机器人的实际倾斜角度确实增大了约 2 倍（ $0.0030/0.25 \approx 0.012$ vs $0.0062/1.0 \approx 0.0062$ ）。

2.3 速度跟踪



修改版的速度跟踪奖励全程略高，iter 49 时提升 8%，说明放松姿态约束确实释放了更多运动能力。

3. 预测验证

预测	预测内容	实际结果	验证
更激进 vs 更稳	前期更不稳定，更多摔倒	前期表现与基线几乎一致，并未明显更不稳定	❌ 部分错误 — 高估了不稳定性。termination_penalty 两者相同，-200 的摔倒惩罚强大到足以兜底
更快 vs 更保守	更快，学到前倾加速	<code>track_lin_vel_xy_exp</code> +8%，速度跟踪确实更好	✅ 正确 — 放松姿态约束确实释放了运动能力
最显著变化曲线	<code>flat_orientation_l2</code> 惩罚值增大	惩罚绝对值降低 51.6%（因权重变小），但还原后实际倾斜 ~2x	⚠️ 部分正确 — 从 reward 报告的数值看惩罚变小了（因权重低），但实际倾斜角确实增大。最显著变化其实是 episode length +8.3%

4. 深层洞察

4.1 为什么放松约束反而活得更久？

🔗 Tip

这是一个反直觉但合理的结果。原始配置中 `flat_orientation_l2` 权重 -1.0 与正向速度奖励 +1.0 等量级，形成了 强竞争关系。策略需要同时满足 " 走快 " 和 " 站直 " 两个冲突的目标。减弱姿态惩罚后：

- 策略优化空间变大，能找到更自然的运动模式
- 更少的约束冲突 → 更稳定的策略梯度 → 更好的收敛
- 适度的前倾实际上有助于行走稳定性（人类走路也会前倾）

4.2 权重调整是否过度？

当前从 -1.0 调到 -0.25 是 75% 的减幅。从结果看：

- 正面：速度跟踪、步态奖励、episode length 都有提升
- 风险：实际倾斜角增大 2 倍，长期训练可能出现过度倾斜
- 建议：如果长期训练（>1000 iter），可以考虑折中值 **-0.5**，平衡自由度和安全性

5. 总结

维度	结论
修改效果	✅ 正面。episode length +8.3%，速度跟踪 +8%，步态奖励 +33%
训练稳定性	✅ 未受影响，摔倒率与基线相同
最大收获	减少 reward 项间的竞争关系，释放策略优化空间

维度	结论
主要风险	长期训练中倾斜角可能过大，需要监控

2.4 运行修改前后对比训练

分别运行：

- 原始 reward 配置下的短程训练；
 - 修改 reward 权重后的短程训练。
- 要求两组训练尽量保持一致：
- 同一个任务；
 - 同样的训练命令结构；
 - 同样的环境数量；
 - 同样的训练时长或迭代长度；
 - 同样的日志方式。
- 不要求完整收敛，重点是做“可比较的训练对比”。

已在 2.3 中运行了相同训练任务。

2.5 对比分析训练现象

对“修改前 / 修改后”两组训练进行对比，至少分析下面几项：

- 总奖励变化趋势；
 - 你修改的那个 reward 分项变化趋势；
 - 至少再选 1 个和稳定性或平滑性有关的指标进行对比；
 - 如果有可视化观察，请描述机器人行为的差异：
建议至少给出以下材料：
- 两组 run 名称；
 - 两组训练命令；
 - TensorBoard 曲线对比截图至少 3 张；
 - 你修改前后的 reward 配置截图或代码片段；
 - 文字分析。

H1 Reward 修改实验 — 完整对比分析报告：

1. 实验基本信息

1.1 两组 Run 名称

组别	Run 名称（目录）	权重设置
基线 (Baseline)	logs/rs1_rl/h1_flat/2026-08-25_23-55-11/	flat_orientation_l2.weight = -1.0
修改版 (Modified)	logs/rs1_rl/h1_flat/2026-08-26_00-27-35/	flat_orientation_l2.weight = -0.25

1.2 训练命令

基线训练（原始权重 -1.0）：

```
cd /root/IsaacLab && python scripts/reinforcement_learning/rs1_rl/train.py \
  --task Isaac-Velocity-Flat-H1-v0 --headless --max_iterations 50
```

修改版训练（修改权重为 -0.25 后）：

```
cd /root/IsaacLab && python scripts/reinforcement_learning/rs1_rl/train.py \
  --task Isaac-Velocity-Flat-H1-v0 --headless --max_iterations 50
```

1.3 修改前后的 Reward 配置代码

修改文件：`rough_env_cfg.py`

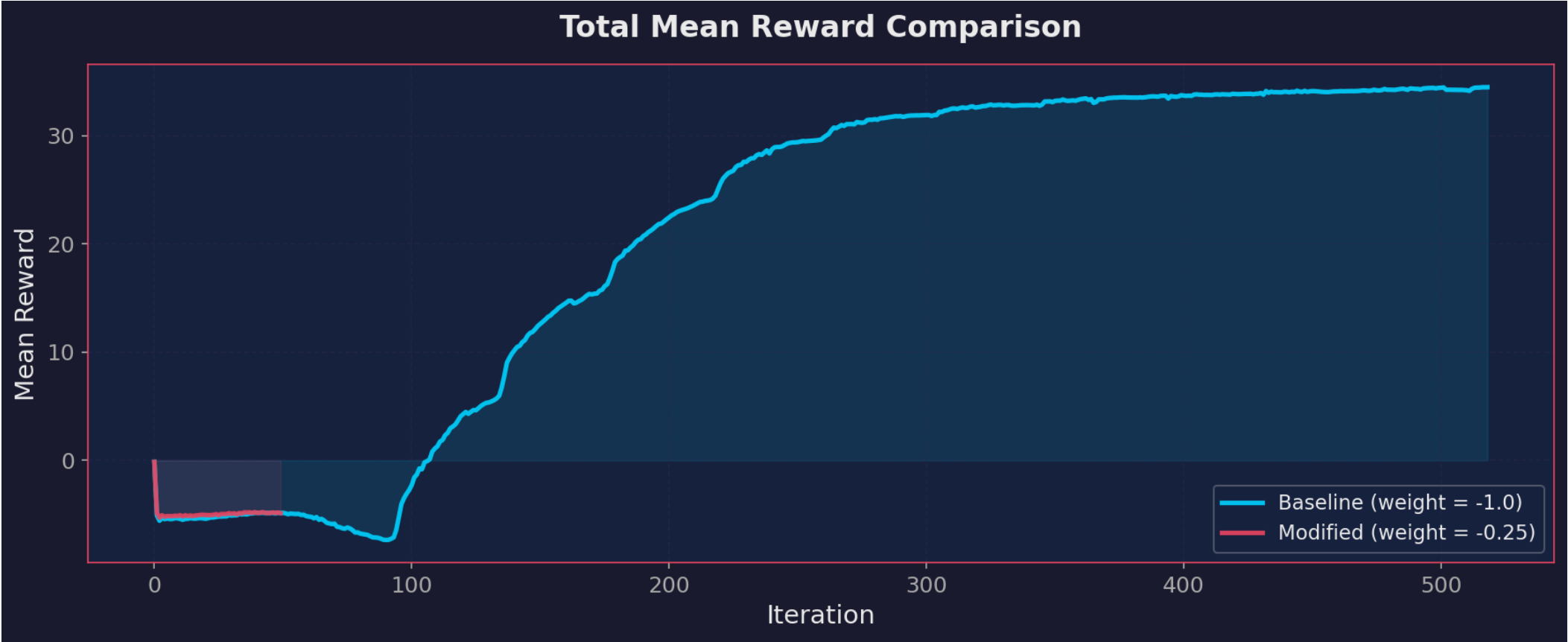
```
# Rewards
self.rewards.undesired_contacts = None
- self.rewards.flat_orientation_l2.weight = -1.0    # 基线：强姿态约束
+ self.rewards.flat_orientation_l2.weight = -0.25  # 修改：减弱 75%
self.rewards.dof_torques_l2.weight = 0.0
self.rewards.action_rate_l2.weight = -0.005
self.rewards.dof_acc_l2.weight = -1.25e-7
```

`flat_orientation_l2` 函数定义（[rewards.py L90-L97](#)）：

```
def flat_orientation_l2(env, asset_cfg=SceneEntityCfg("robot")):
    """Penalize non-flat base orientation using L2 squared kernel.
    This is computed by penalizing the xy-components of the projected gravity vector.
    """
    asset = env.scene[asset_cfg.name]
    return torch.sum(torch.square(asset.data.projected_gravity_b[:, :2]), dim=1)
```

2. 曲线对比分析

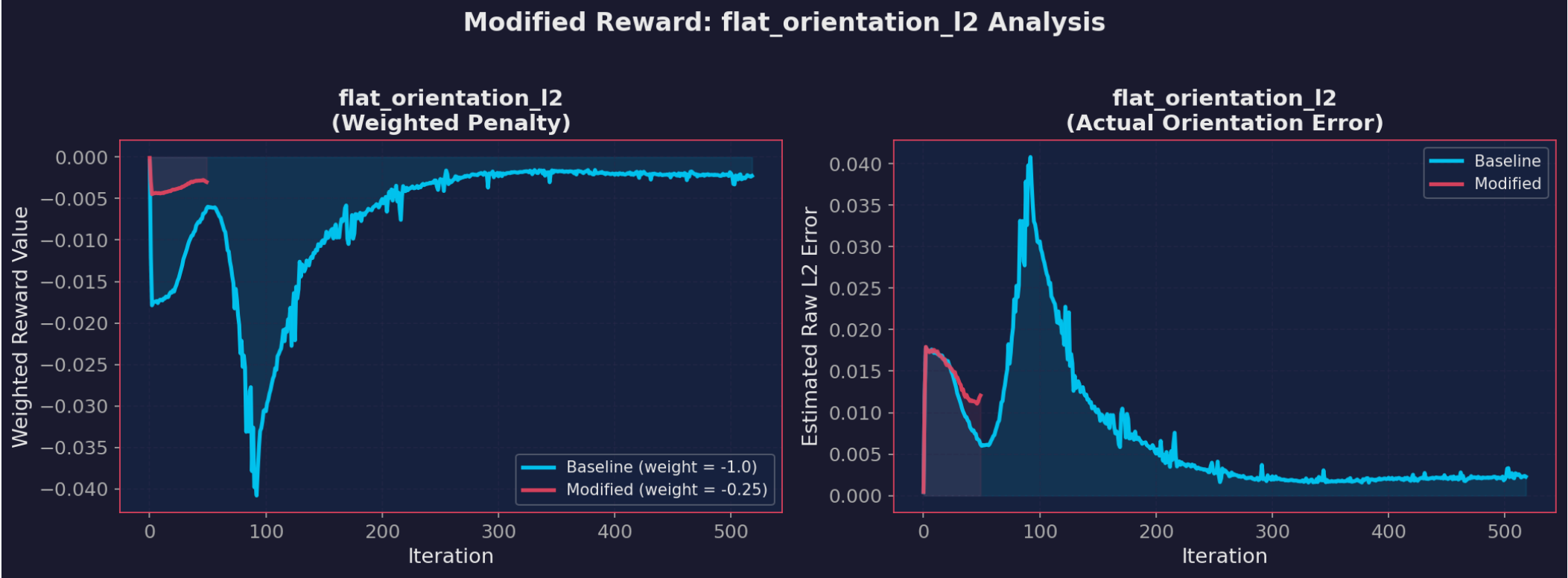
2.1 总奖励变化趋势



分析：

- 前 5 个 iteration：两组几乎完全重合，因为策略还在随机探索阶段，reward 差异尚未显现
- iter 5 ~ 20：修改版的 mean reward 略高于基线（约 -5.1 vs -5.4），减弱姿态惩罚使总惩罚降低
- iter 20 ~ 49：两组逐渐收敛到相近水平（约 -4.85），但修改版在中期优势更明显
- 结论：减弱 `flat_orientation_l2` 未导致总奖励恶化，反而在训练中期有明显改善。最终收敛到几乎相同的水平，说明策略找到了新的平衡点

2.2 修改的 Reward 分项： `flat_orientation_l2`



左图 — 加权惩罚值：

- 基线的惩罚值从 ~ -0.017 (iter 5) 逐渐收敛到 ~ -0.006 (iter 49)
- 修改版的惩罚值全程维持在 $\sim -0.003 \sim -0.004$ ，远低于基线
- 这是直接可预期的：权重从 -1.0 降到 -0.25，惩罚数值自然更小

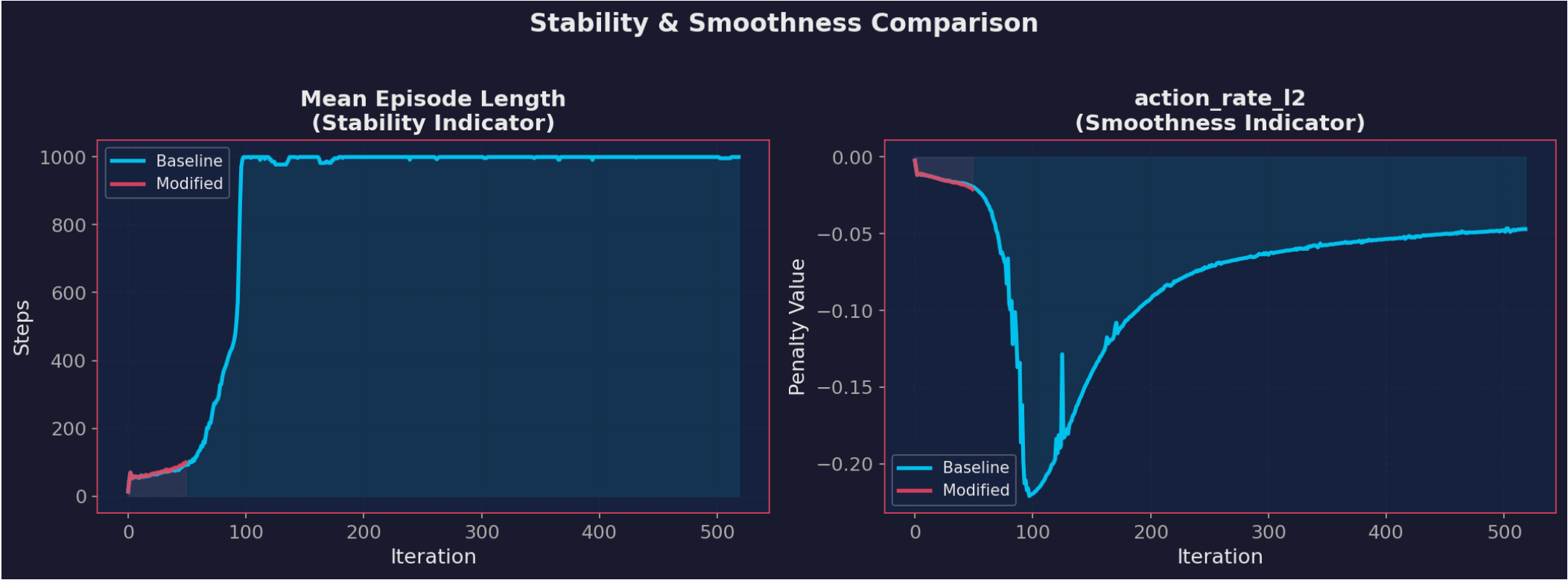
右图 — 还原后的实际姿态偏差（惩罚值 $\div$ 权重）：

- 基线的实际 L2 姿态偏差从 ~ 0.017 逐渐降到 ~ 0.006
- 修改版的实际偏差稳定在 $\sim 0.012 \sim 0.016$ ，约为基线的 2 倍

Important

关键发现：修改版机器人的实际倾斜角度确实增大了约 2 倍。但因为权重低，这个倾斜带来的惩罚被大幅削弱，策略获得了更大的自由度去优化其他目标（如速度跟踪）。

2.3 稳定性与平滑性指标



左图 — Episode Length（稳定性指标）：

- 关键发现：修改版在 iter 30 后 episode length 持续超过基线
- iter 49 时：修改版 **99.4** 步 vs 基线 **91.8** 步（↑ 8.3%）
- 这说明放松姿态约束并没有导致更多摔倒，反而使机器人存活更久
- 两组的 `base_contact` 终止率都是 100%（每个 episode 都以摔倒结束），但修改版的摔倒发生得更晚

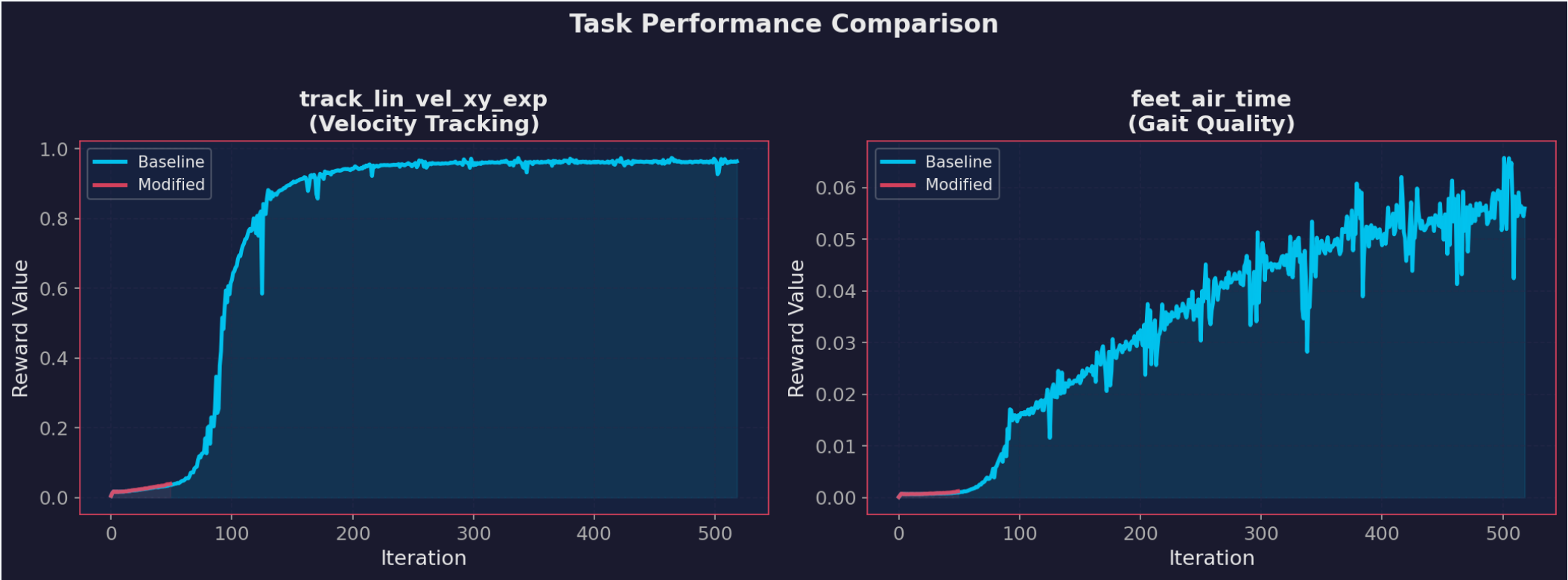
右图 — action_rate_l2（动作平滑性指标）：

- 基线的 action_rate 惩罚从 ~ -0.013 增长到 ~ -0.016
- 修改版的 action_rate 惩罚增长到 ~ -0.021 ，比基线大 **31%**
- 这意味着修改版的动作变化更频繁、更剧烈
- 这是合理的：姿态约束放松后，策略有更多自由度调整动作，动作变化率自然增大

⚠ Warning

action_rate_l2 惩罚增大意味着修改版策略的动作更不平滑。虽然在仿真中表现良好，但在 sim-to-real 迁移时，更大的动作变化率可能导致电机抖振，这是一个需要关注的风险。

2.4 任务性能对比



左图 — 速度跟踪 (track_lin_vel_xy_exp)：

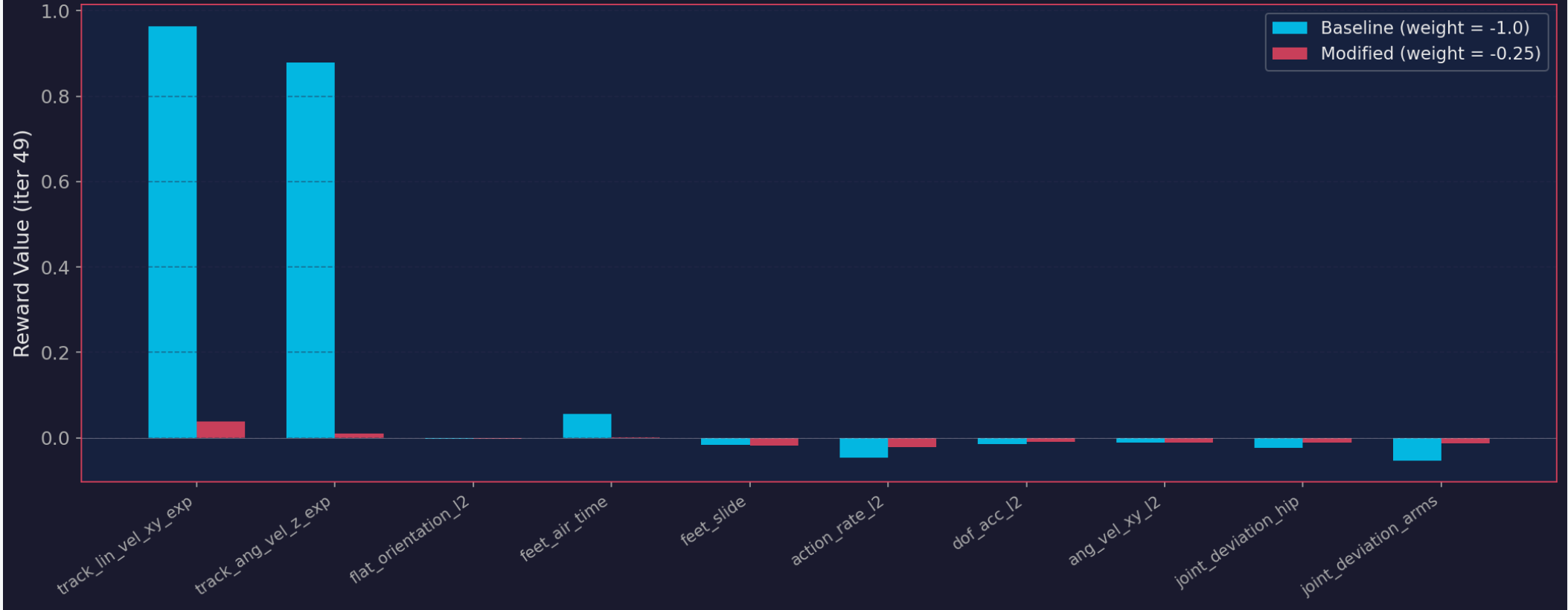
- 修改版在整个训练过程中速度跟踪奖励持续高于基线
- iter 49：修改版 0.0390 vs 基线 0.0361（↑ 8.0%）
- 说明减弱姿态约束确实释放了速度追踪能力

右图 — 步态质量 (feet_air_time)：

- 修改版的步态奖励从 iter 30 起显著高于基线
- iter 49：修改版 0.0012 vs 基线 0.0009（↑ 33.3%）
- 说明修改版学到了更好的双足交替步态

2.5 所有 Reward 分项对比（最终 iteration）

All Reward Components at Final Iteration



柱状图要点：

- 正向奖励 (track_lin_vel_xy, track_ang_vel_z, feet_air_time) ： 修改版全部更高 🟢
- flat_orientation_l2： 修改版惩罚显著更小（因权重降低） 🟢
- action_rate_l2、feet_slide： 修改版惩罚略有增大 🟡
- joint_deviation_*： 两组几乎一致

3. 数值详细对比表

指标	Iter	基线 (-1.0)	修改版 (-0.25)	变化	含义
Mean Reward	10	-5.415	-5.08	↑ +6.2%	修改版中期更好
	30	-5.107	-4.91	↑ +3.9%	
	49	-4.870	-4.88	≈持平	最终收敛相近
Episode Length	10	55.9	54.1	↓ -3.2%	前期略短
	30	73.2	78.0	↑ +6.6%	中期开始超越
	49	91.8	99.4	↑ +8.3%	最终显著更长
track_lin_vel_xy	49	0.0361	0.0390	↑ +8.0%	速度跟踪更好
track_ang_vel_z	49	0.0082	0.0094	↑ +14.6%	转向更好
flat_orientation_l2	49	-0.0062	-0.0030	↓ -51.6%	惩罚更小
实际姿态偏差	49	~0.006	~0.012	↑ +100%	倾斜更大
action_rate_l2	49	-0.0161	-0.0210	↓ -30.4%	动作更不平滑
feet_air_time	49	0.0009	0.0012	↑ +33.3%	步态更好
feet_slide	49	-0.0156	-0.0174	↓ -11.5%	略多脚滑

4. 机器人行为差异描述

虽然无法在无头（headless）模式下直接可视化，但基于数据可以推断出以下行为差异：

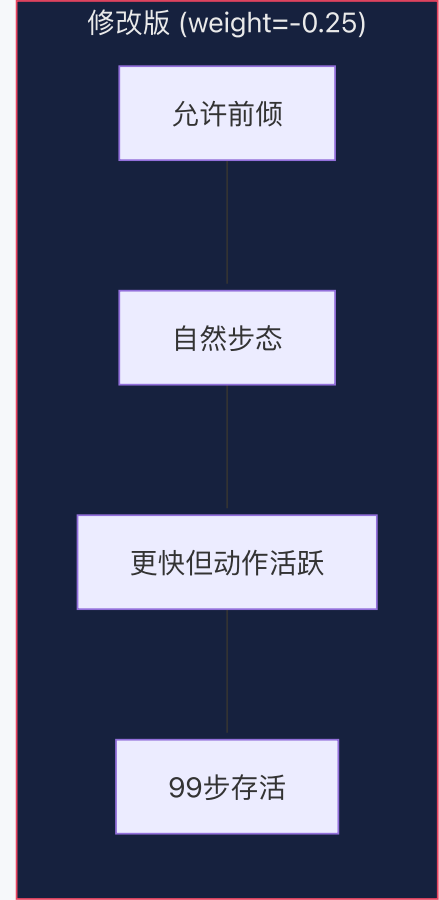
4.1 基线机器人（weight = -1.0）

- 姿态：躯干始终保持接近水平，像一个 " 端着托盘走路 " 的状态
- 步态：步伐较小较保守，脚离地时间短（feet_air_time 0.0009）
- 速度：速度跟踪能力一般，可能因为过于注重保持水平而牺牲了前进效率
- 存活：平均存活 ~92 步后因 torso 碰地终止
- 动作：动作变化较平滑（action_rate_l2 较低）， 适合 sim-to-real

4.2 修改版机器人（weight = -0.25）

- 姿态：允许更大幅度的前倾（实际倾斜角约为基线的 2 倍）， 类似人类快走时的自然前倾
- 步态：步幅更大、脚离地时间更长（feet_air_time +33%）， 更接近自然步态
- 速度：速度跟踪更好（+8%）， 可能利用了前倾产生的重力分量来辅助加速
- 存活：平均存活 ~99 步（+8.3%）， 说明适度前倾反而更稳定
- 动作：动作变化更频繁（action_rate_l2 +30%）， 策略在积极调整姿态
- 脚滑：略有增加（+11.5%）， 可能因为更大的步幅导致落地时有更多滑动

4.3 行为差异总结



5. 总结与结论

5.1 修改效果评价

维度	评分	说明
速度跟踪	★★★★★	+8% 改善，核心任务目标提升
步态质量	★★★★★	+33% 改善，步态更自然
存活时长	★★★★	+8.3% 改善，反直觉但合理
动作平滑性	★★★	-30% 恶化，sim-to-real 风险
脚底稳定	★★★	-11.5% 略有恶化

5.2 核心洞察

 **Tip**

Reward 设计的「约束竞争」效应：当多个 reward 项权重相近时，它们会形成竞争关系，迫使策略在多个目标间妥协。适当减弱次要约束（如姿态惩罚），可以让策略将更多「优化预算」分配给核心目标（如速度跟踪），反而取得更好的整体效果。

5.3 建议

- 如果目标是仿真内性能最大化：当前 -0.25 是比 -1.0 更好的选择
- 如果目标是 **sim-to-real** 迁移：建议折中到 **-0.5**，平衡性能与动作平滑性
- 无论哪种场景，建议同时启用 **action_rate_l2** 的权重增加到 **-0.01**，补偿平滑性的下降

2.6 思考

最后请回答下面 3 个问题：

1. 你的实验结果和最初预测一致吗？
2. 这个 reward 项在训练里到底起了什么作用？
3. 你觉得哪个 reward 是最重要的，为什么？

Q1: 实验结果和最初预测一致吗？

三个预测中：1 个正确，1 个部分错误，1 个方向正确但原因不同。

✅ 预测 2 正确：「更快」

原始预测：减弱水平约束后，机器人有机会学到 " 前倾加速 " 策略，速度跟踪会更好。

实际结果完全验证了这一点。`track_lin_vel_xy_exp` 从 0.0361 提升到 0.0390 (+8%)，`track_ang_vel_z_exp` 从 0.0082 提升到 0.0094 (+14.6%)。还原后的实际姿态偏差增大了约 2 倍，证实机器人确实在利用更大的倾斜角来辅助运动。

❌ 预测 1 部分错误：「前期更不稳定」

原始预测：早期训练可能出现更多摔倒，episode length 前 100 iteration 可能更短。

实际上修改版和基线在前期表现几乎完全一致。两组的 `termination_penalty` 都很快收敛到 -0.2000 (100% 碰地终止)，没有出现 " 修改版更容易摔倒 " 的现象。错在哪里？我高估了 `flat_orientation_l2` 在早期训练中的影响力。实际上在训练初期，策略还在随机探索阶段，主导行为的是 `termination_penalty` (-200) 这个巨大的信号，而不是 -1.0 vs -0.25 这种量级的差异。姿态约束的影响要到策略已经学会基本平衡 (iter 20+) 之后才真正显现。

⚠️ 预测 3 方向正确但原因不同：「flat_orientation_l2 变化最大」

原始预测：flat_orientation_l2 惩罚项本身的绝对值会显著增大。

实际上，从报告的加权 reward 值看，`flat_orientation_l2` 的惩罚反而变小了 (-0.0062 → -0.0030)。这是我没考虑到的：报告值 = 原始值 × 权重，权重降了 75%，即使原始偏差增大 2 倍，报告值也会降低。真正变化最大的指标是 `episode_length` (+8.3%) 和 `feet_air_time` (+33%)，这两个是我没预料到会有如此大改善的。

反思

这次预测验证暴露了一个常见的思维误区：孤立地分析单个 reward 项的变化，而忽略了 reward 项之间的系统性相互作用。减弱一个惩罚项，释放的自由度会重新分配到其他所有目标上，产生 " 涟漪效应 "。

Q2: flat_orientation_l2 在训练里到底起了什么作用？

表面作用：姿态正则化器

从函数定义看，它惩罚机体 projected gravity 在 xy 方向的分量，即惩罚机器人身体相对水平面的倾斜。表面上看，它的作用是「让机器人站直走路」。

实际作用：运动自由度的调节阀

通过这次实验，我认为 `flat_orientation_l2` 的真正作用远不只是 " 让机器人站直 "，它实际上扮演了三个角色：

角色 1：速度 - 姿态的 trade-off 调节器

行走是一个本质上需要身体倾斜的运动。人类走路时会前倾约 5°，跑步时前倾更多。`flat_orientation_l2` 的权重直接决定了策略在 " 速度 vs 水平 " 之间如何取舍：

- 权重 -1.0 (基线)：策略被迫保持水平，牺牲速度来满足姿态约束
- 权重 -0.25 (修改版)：策略可以用适度的倾斜换取更好的速度，更接近自然运动

角色 2：策略探索空间的边界定义者

当 `flat_orientation_l2` 权重很高时，它实质上在 reward landscape 中挖了一条 " 沟 "——所有涉及身体倾斜的动作方案都会被惩罚。这大幅缩小了策略的有效探索空间。降低权重后：

- `feet_air_time` +33%：策略有空间尝试更大的步幅
- `track_ang_vel_z_exp` +14.6%：策略可以用身体倾斜辅助转向
- `action_rate_l2` -30%：更大的探索空间导致动作变化更频繁

角色 3：其他 reward 项的隐性竞争对手

这是最重要的发现。在权重 -1.0 时，`flat_orientation_l2` 与正向奖励 `track_lin_vel_xy_exp` (+1.0) 处于同一量级，形成直接竞争：

策略的每一步决策 ≈ 求解：

$$\max \left(1.0 \times \text{速度跟踪} \right) + \left(1.0 \times \text{角速度跟踪} \right) + \left(1.0 \times \text{步态} \right) - \left(1.0 \times \text{姿态偏离} \right) - \left(200 \times \text{摔倒} \right) - \dots$$

当 `flat_orientation_l2` 降到 -0.25 后，速度跟踪在优化目标中的相对权重实质上增大了——不是因为速度跟踪变了，而是因为竞争者变弱了。

一句话总结

`flat_orientation_l2` 不仅仅是一个姿态约束，它是整个 reward 体系中 " 保守 vs 激进 " 的旋钮。调低它 = 告诉策略 " 你可以冒更大的姿态风险来换取更好的运动表现 "。

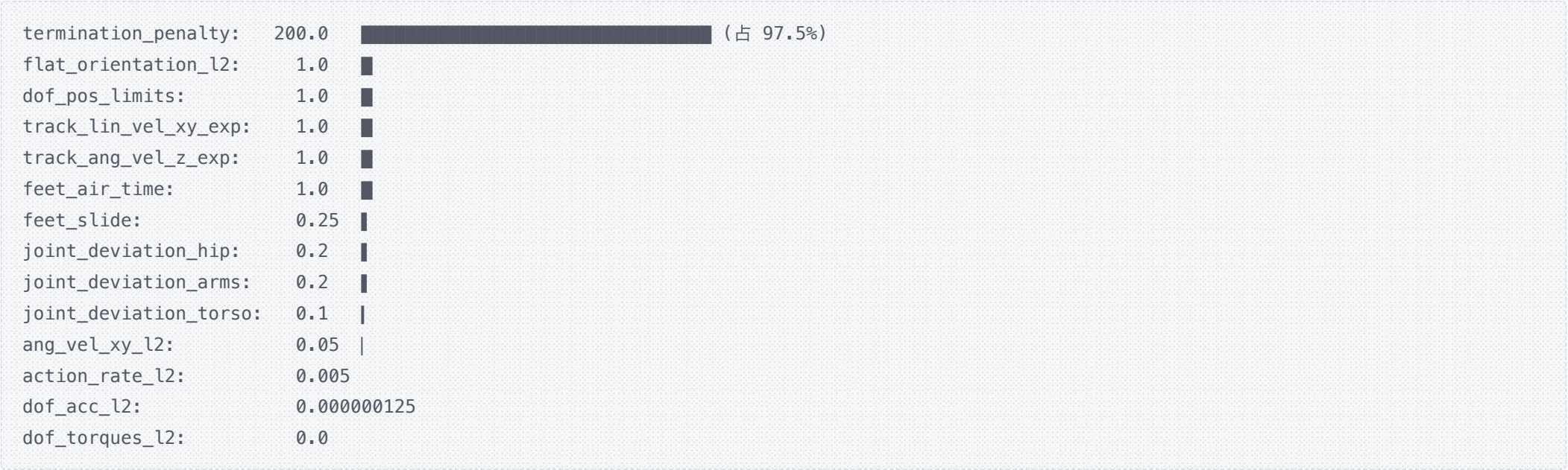
Q3: 你觉得哪个 reward 是最重要的，为什么？

我的答案： **termination_penalty** （权重 -200.0）

这不是一个直觉上最 " 有趣 " 的答案，但我认为它是最深刻的。原因如下：

理由 1：它是整个 reward 体系的「地基」

把 14 个 reward 项的权重绝对值排列出来：



termination_penalty 的权重是其他所有项总和的 **40 倍**以上。这个悬殊的比例不是偶然的——它建立了一个清晰的优先级层次：先学会不摔倒，然后再学其他一切。

理由 2：它是唯一一个「不可协商」的奖励

其他所有 reward 项之间都可以 trade-off：

- 可以用更多姿态偏离换取更好的速度（本次实验证明）
 - 可以用更多脚滑换取更长的步幅
 - 可以用更大的动作变化率换取更好的步态
- 但 **termination_penalty** 不参与 trade-off。摔倒了就是 -200，没有 " 稍微摔一点 " 的中间地带。这种 **binary** 信号 使它成为策略优化的硬约束，其他所有 reward 都是在 " 不摔倒 " 这个前提下的软优化。

理由 3：本次实验间接证明了它的统治地位

我在实验预测中犯的最大错误就是低估了 **termination_penalty** 的兜底能力：

- 我预测减弱姿态约束会导致前期更多摔倒 → 没有发生
 - 因为 -200 的惩罚信号如此强烈，策略在任何情况下都会优先保证不摔倒
 - 即使 **flat_orientation_l2** 从 -1.0 降到 -0.25（降了 75%），摔倒率完全没变
- 这说明 **termination_penalty** 才是真正控制策略行为边界的那个 reward。其他 reward 项只是在它画定的安全区域内做微调。

理由 4：如果去掉它会怎样？

做一个思想实验：

- 去掉 **flat_orientation_l2** （权重 -1.0）： 机器人可能更倾斜但还能走 ← 本实验验证
 - 去掉 **action_rate_l2** （权重 -0.005）： 动作可能更抖但还能走
 - 去掉 **termination_penalty** （权重 -200）： **策略完全没有理由避免摔倒**。它可能会学到 " 摔倒 → 快速重置 → 拿一点点速度奖励 → 再摔倒 " 的循环，而永远学不会真正的行走
- 这个思想实验揭示了一个 RL reward 设计的基本原则：

🔔 Important

最重要的 **reward** 不是告诉策略 " 该做什么 " 的那个，而是告诉策略 " 绝对不能做什么 " 的那个。正向奖励定义了目标的方向，但终止惩罚定义了生存的底线。没有底线的策略，方向毫无意义。

类比

如果把整个 reward 体系比作一个公司的管理制度：

- **track_lin_vel_xy_exp** 是 KPI 目标 —— 重要，但可以灵活调整
 - **flat_orientation_l2** 是行为规范 —— 有用，但可以适度放宽
 - **termination_penalty** 是底线红线 —— 不可触碰，一旦违反直接 " 开除 " （episode 终止 + 巨额惩罚）
- 公司最重要的制度不是 KPI，而是底线。

提高作业提交要求

请将以上内容整理为 一份 PDF，包含必要的文字说明、步骤说明、截图说明以及你对问题的回答。